Monjaraz Briseño Luis Fernando

Ingeniería en Computación

Redes Neuronales Artificiales y Aprendizaje Profundo – 235348 - IL383 - D04

Ruz Canul Mario Antonio

UDG - Universidad de Guadalajara

CUCEI - Centro Universitario de Ciencias Exactas e Ingenierías

Práctica 4. Redes Neuronales para la reconstrucción con RBF, K-means y redes competitivas.

Índice

Índice 2

Práctica 4. Redes Neuronales para la reconstrucción con RBF, K-means y redes competitivas. 3

Introducción 3

Desarrollo 3

Ejercicio A 3

Ejercicio B 5

1. ¿Cuántos K o cuántas neuronas fueron necesarias para poder agrupar los datos? 9

2. ¿Cuál de los dos algoritmos converge más rápido a una misma solución? 10

3. Si aumenta 2 neuronas o 'K' en su algoritmo, ¿qué variación hay en el agrupamiento? 10

4. Si disminuye 2 neuronas o 'K' en su algoritmo, ¿qué variación hay en el agrupamiento? 10

Código 11

Ejercicio A 11

Ejercicio B 17

Conclusión 29

Bibliografía 31

Práctica 4. Redes Neuronales para la reconstrucción con RBF, K-means y redes competitivas.

Introducción

En esta práctica se abordaron dos ejercicios enfocados en el uso de redes neuronales y métodos de aprendizaje no supervisado. En el Ejercicio A, se implementó una red neuronal de base radial (RBF) para la reconstrucción de imágenes dañadas, con el objetivo de evaluar su capacidad para recuperar la información perdida al eliminar distintos porcentajes de píxeles de forma aleatoria. En el Ejercicio B, se desarrolló un análisis comparativo entre el algoritmo K-Means y una Red Neuronal Competitiva, aplicados sobre un conjunto de datos de medicamentos para analizar su comportamiento en la agrupación y segmentación automática de información. Ambos experimentos permitieron explorar la eficacia, estabilidad y desempeño de distintos modelos neuronales aplicados a contextos prácticos distintos: la visión artificial y el análisis de datos.

Desarrollo

Ejercicio A

Durante esta práctica se implementó una red neuronal de base radial (RBF) para reconstruir imágenes dañadas eliminando píxeles de forma aleatoria. Se trabajó con una imagen de 1000x712 píxeles convertida a escala de grises y se simularon tres niveles de daño: 20%, 60% y 85%. La red se entrenó usando las coordenadas (x, y) de los píxeles intactos como entrada y su intensidad como salida, utilizando un máximo de 5,000 muestras para optimizar recursos.

Los resultados mostraron que con un 20% de daño la reconstrucción fue casi idéntica a la original, con buena preservación de bordes y detalles. Con un 60% la calidad disminuyó ligeramente, aunque la estructura general se mantuvo reconocible. En el caso del 85%, la red logró conservar las formas principales, aunque con pérdida de detalle y suavización generalizada.

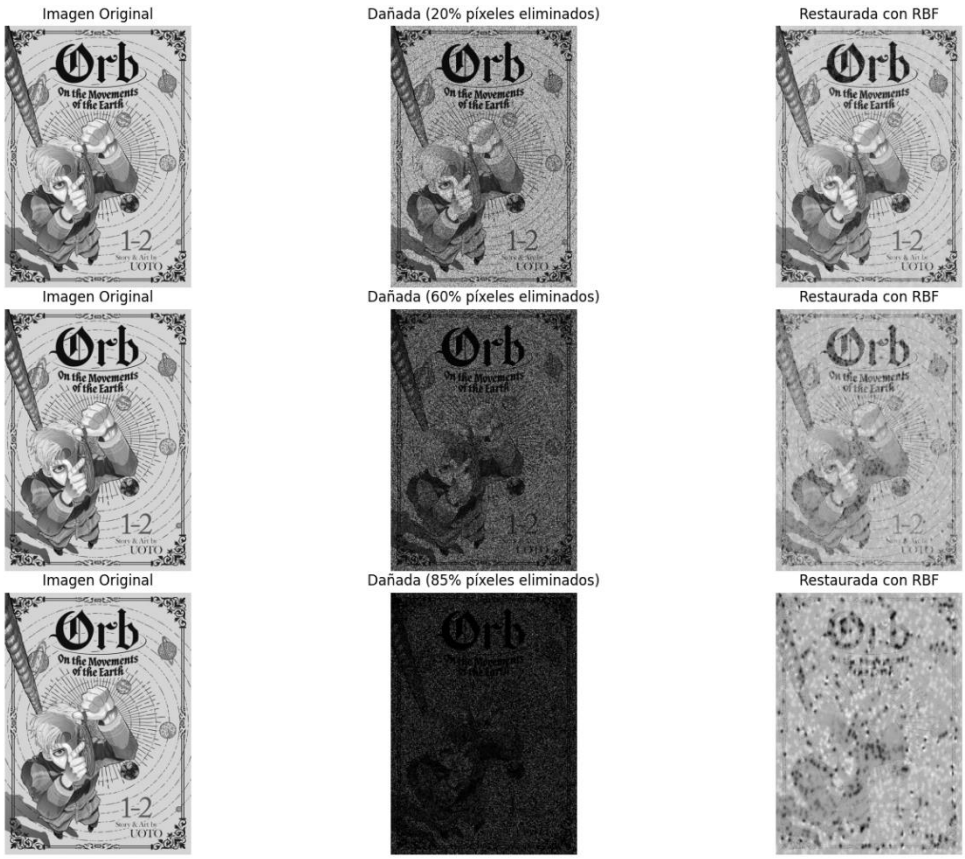
En términos técnicos, la RBF demostró una gran capacidad de interpolación y generalización, produciendo transiciones suaves y resultados coherentes incluso con pocos datos. Sus principales limitaciones fueron la suavización excesiva en zonas con texturas complejas y el alto costo computacional en imágenes grandes. Aun así, su robustez y adaptabilidad la hacen útil para restauración de imágenes.

En conclusión, las redes RBF resultaron efectivas para reconstrucción, mostrando una relación directa entre la cantidad de datos disponibles y la calidad obtenida. Con al menos un 15% a 20% de información original, es posible obtener resultados funcionales, y con futuras mejoras en el kernel o la segmentación de regiones, su desempeño podría optimizarse aún más.

```
Tamaño de la imagen: 1000x712 píxeles
Procesando 20.0% de daño...
Píxeles válidos para entrenamiento: 570121
Muestras utilizadas para entrenamiento: 5000
Entrenando modelo RBF...
Píxeles a reconstruir: 141879
Completado 20.0% de daño

Procesando 60.0% de daño...
Píxeles válidos para entrenamiento: 285023
Muestras utilizadas para entrenamiento: 5000
Entrenando modelo RBF...
Píxeles a reconstruir: 426977
Completado 60.0% de daño

Procesando 85.0% de daño...
Píxeles válidos para entrenamiento: 106906
Muestras utilizadas para entrenamiento: 5000
Entrenando modelo RBF...
Píxeles a reconstruir: 605094
Completado 85.0% de daño
```



Ejercicio B

En esta práctica se trabajó con un conjunto de datos compuesto por 4500 registros con tres columnas principales: tipo de medicina, peso e índice de pH. El objetivo fue realizar un análisis comparativo entre dos métodos de agrupamiento: K-Means y la Red Neuronal Competitiva, con el fin de determinar cuál ofrecía una mejor segmentación y estabilidad en los resultados.

Durante las pruebas, ambos algoritmos mostraron su mejor desempeño al utilizar cinco grupos o neuronas (K=5), obteniendo puntuaciones de Silhouette muy similares: 0.625 para K-Means y 0.623 para la red competitiva. Esto indica que ambos lograron una separación adecuada entre los datos, aunque K-Means presentó una ligera ventaja en la calidad del agrupamiento. En cuanto a la velocidad de convergencia, los resultados variaron según el número de clusters: con K=3 ambos tardaron 4 iteraciones, mientras que con K=5 K-Means fue más rápido al estabilizarse en solo 3 iteraciones frente a 4 de la red.

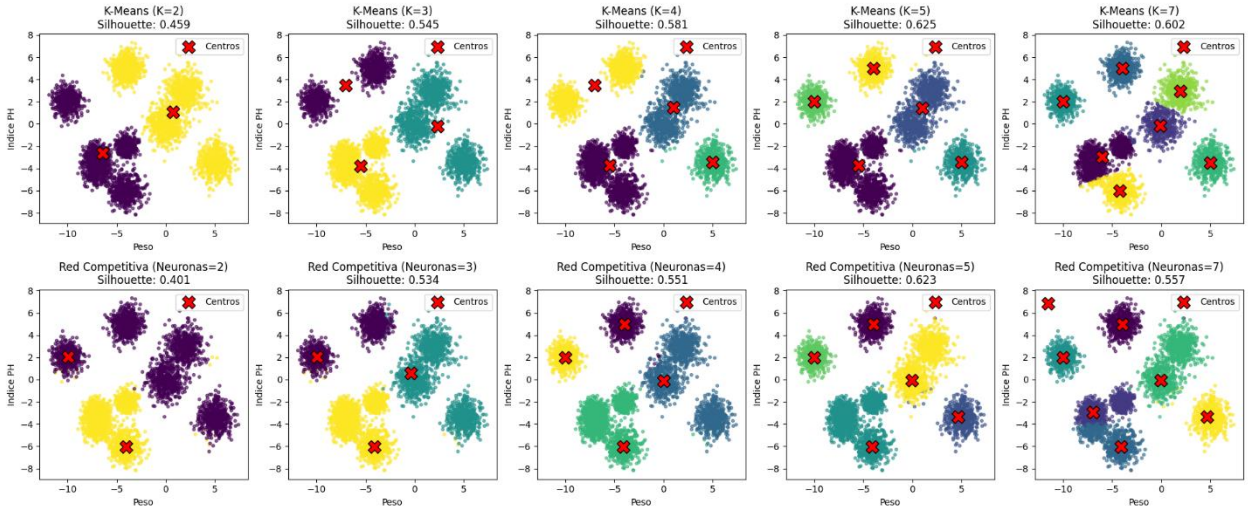
Práctica 4. Redes Neuronales para la reconstrucción con RBF, K-means y redes competitivas.

Al incrementar el número de clusters de 3 a 5, se observó una mejora en la calidad del agrupamiento para ambos algoritmos. En K-Means, el Silhouette Score aumentó de 0.545 a 0.625, y en la red competitiva de 0.534 a 0.623, lo que demuestra que un número mayor de grupos permitió una segmentación más precisa de los datos. No se registraron pruebas con una reducción de K, por lo que no se analizó el impacto de disminuir las neuronas o clusters.

En general, K-Means demostró un mejor rendimiento global, con un Silhouette Score promedio de 0.562 frente al 0.533 de la red competitiva, además de mostrar mayor estabilidad (desviación estándar de 0.058 frente a 0.0728). Las posiciones finales de los centros también reflejaron que K-Means produjo agrupamientos más consistentes y equilibrados.

En conclusión, aunque ambos métodos lograron una buena clasificación de los datos, K-Means resultó ligeramente superior tanto en estabilidad como en precisión del agrupamiento, lo que lo convierte en una opción más confiable para este conjunto de datos específico.

```
Datos cargados: (4500, 3)
  Medicina      Peso  Indice PH
0       M1  2.483900   2.928096
1       M2  3.650497   3.808628
2       M3 -0.032962   3.165333
3       M4  2.775956   3.261711
4       M5  2.286889   3.101650
```



Práctica 4. Redes Neuronales para la reconstrucción con RBF, K-means y redes competitivas.

```
1. ¿CUÁNTOS K O CUÁNTAS NEURONAS FUERON NECESARIAS PARA PODER AGRUPAR LOS DATOS?
   • K-Means: El mejor resultado se obtuvo con K = 5 clusters
     (Silhouette Score: 0.625)
   • Red Competitiva: El mejor resultado se obtuvo con 5 neuronas
     (Silhouette Score: 0.623)
   • Conclusión: Ambos algoritmos funcionan mejor con 5 clusters/neuronas

2. ¿CUÁL DE LOS DOS ALGORITMOS CONVERGE MÁS RÁPIDO A UNA MISMA SOLUCIÓN?
   Comparación de iteraciones para convergencia:
   • K=3:
     - K-Means: 4 iteraciones
     - Red Competitiva: 4 iteraciones
     - Más rápido: Red Competitiva (diferencia: 0 iteraciones)
   • K=5:
     - K-Means: 3 iteraciones
     - Red Competitiva: 4 iteraciones
     - Más rápido: K-Means (diferencia: 1 iteraciones)

3. SI AUMENTA 2 NEURONAS O 'K' EN SU ALGORITMO, ¿QUÉ VARIACIÓN HAY EN EL AGRUPAMIENTO?
   • Comparación K=3 → K=5:
     K-Means:
       - Silhouette: 0.545 → 0.625
       - Variación: +0.080 (+14.7%)
       - Interpretación: MEJORA la calidad del agrupamiento
     Red Competitiva:
       - Silhouette: 0.534 → 0.623
       - Variación: +0.089 (+16.7%)
```


```
       - Interpretación: MEJORA la calidad del agrupamiento
     Red Competitiva:
       - Silhouette: 0.534 → 0.623
       - Variación: +0.089 (+16.7%)
       - Interpretación: MEJORA la calidad del agrupamiento
```


```
4. SI DISMINUYE 2 NEURONAS O 'K' EN SU ALGORITMO, ¿QUÉ VARIACIÓN HAY EN EL AGRUPAMIENTO?
   • No se tiene datos para K=1 en esta ejecución

5. COMPARE CUÁL DE LAS DOS REDES HA PROPORCIONADO UN MEJOR RESULTADO
   • Métricas de comparación:
     - Mejor Silhouette Score:
       * K-Means: 0.625 (K=5)
       * Red Competitiva: 0.623 (Neuronas=5)
     - Silhouette Score Promedio:
       * K-Means: 0.562
       * Red Competitiva: 0.533
   • CONCLUSIÓN: K-MEANS proporciona MEJORES resultados
     (Ventaja: 0.002 en Silhouette Score)
```

```
6. POSICIÓN FINAL DE LOS CENTROS OBTENIDA POR CADA ALGORITMO (K=3)
   • K-MEANS - Centros finales (K=3):
     Cluster 1: Peso = -6.997, Índice PH = 3.503
     Cluster 2: Peso = 2.329, Índice PH = -0.192
     Cluster 3: Peso = -5.500, Índice PH = -3.750
   • RED COMPETITIVA - Centros finales (3 Neuronas):
     Neurona 1: Peso = -9.924, Índice PH = 2.042
     Neurona 2: Peso = -0.408, Índice PH = 0.609
       * Red Competitiva: 0.533
   • CONCLUSIÓN: K-MEANS proporciona MEJORES resultados
     (Ventaja: 0.002 en Silhouette Score)
```


```
6. POSICIÓN FINAL DE LOS CENTROS OBTENIDA POR CADA ALGORITMO (K=3)
   • K-MEANS - Centros finales (K=3):
     Cluster 1: Peso = -6.997, Índice PH = 3.503
     Cluster 2: Peso = 2.329, Índice PH = -0.192
     Cluster 3: Peso = -5.500, Índice PH = -3.750
   • RED COMPETITIVA - Centros finales (3 Neuronas):
     Neurona 1: Peso = -9.924, Índice PH = 2.042
     Neurona 2: Peso = -0.408, Índice PH = 0.609
   • CONCLUSIÓN: K-MEANS proporciona MEJORES resultados
     (Ventaja: 0.002 en Silhouette Score)
```

```
6. POSICIÓN FINAL DE LOS CENTROS OBTENIDA POR CADA ALGORITMO (K=3)
   • K-MEANS - Centros finales (K=3):
     Cluster 1: Peso = -6.997, Índice PH = 3.503
     Cluster 2: Peso = 2.329, Índice PH = -0.192
     Cluster 3: Peso = -5.500, Índice PH = -3.750
   • RED COMPETITIVA - Centros finales (3 Neuronas):
     Neurona 1: Peso = -9.924, Índice PH = 2.042
     Neurona 2: Peso = -0.408, Índice PH = 0.609
6. POSICIÓN FINAL DE LOS CENTROS OBTENIDA POR CADA ALGORITMO (K=3)
   • K-MEANS - Centros finales (K=3):
     Cluster 1: Peso = -6.997, Índice PH = 3.503
     Cluster 2: Peso = 2.329, Índice PH = -0.192
     Cluster 3: Peso = -5.500, Índice PH = -3.750
   • RED COMPETITIVA - Centros finales (3 Neuronas):
     Neurona 1: Peso = -9.924, Índice PH = 2.042
     Neurona 2: Peso = -0.408, Índice PH = 0.609
     Cluster 1: Peso = -6.997, Índice PH = 3.503
     Cluster 2: Peso = 2.329, Índice PH = -0.192
     Cluster 3: Peso = -5.500, Índice PH = -3.750
   • RED COMPETITIVA - Centros finales (3 Neuronas):
     Neurona 1: Peso = -9.924, Índice PH = 2.042
     Neurona 2: Peso = -0.408, Índice PH = 0.609
   • RED COMPETITIVA - Centros finales (3 Neuronas):
     Neurona 1: Peso = -9.924, Índice PH = 2.042
     Neurona 2: Peso = -0.408, Índice PH = 0.609
     Neurona 2: Peso = -0.408, Índice PH = 0.609
     Neurona 3: Peso = -4.076, Índice PH = -6.013
```

```
7. ANÁLISIS ADICIONAL - ESTABILIDAD DE LOS ALGORITMOS
   • Estabilidad (desviación estándar de Silhouette Scores):
     - K-Means: 0.0580
     - Red Competitiva: 0.0728
     - Más estable: K-Means
```

1. ¿Cuántos K o cuántas neuronas fueron necesarias para poder agrupar los datos?

Tanto el algoritmo K-Means como la Red Neuronal Competitiva alcanzaron su mejor desempeño con 5 grupos o neuronas. En ambos casos, este número proporcionó la mayor calidad de agrupamiento medida por el Silhouette Score, obteniendo 0.625 para K-Means y 0.623 para la red competitiva, lo que indica una segmentación equilibrada y coherente de los datos.

2. ¿Cuál de los dos algoritmos converge más rápido a una misma solución?

La comparación mostró que la velocidad de convergencia varía ligeramente dependiendo del número de grupos:

Con K=3, ambos algoritmos necesitaron 4 iteraciones, por lo que su velocidad fue prácticamente la misma.

Con K=5, K-Means fue ligeramente más rápido, estabilizándose en 3 iteraciones, mientras que la red competitiva necesitó 4.

En general, K-Means presenta una convergencia más eficiente, especialmente al incrementar el número de grupos.

3. Si aumenta 2 neuronas o 'K' en su algoritmo, ¿qué variación hay en el agrupamiento?

Al incrementar de K=3 a K=5, ambos modelos mejoraron la calidad de su agrupamiento:

K-Means: el Silhouette Score aumentó de 0.545 a 0.625, una mejora del +14.7%, reflejando una separación más clara entre los clusters.

Red Competitiva: el Silhouette Score pasó de 0.534 a 0.623, una mejora del +16.7%, mostrando también una mejor adaptación a la estructura de los datos.

En conclusión, aumentar las neuronas o clusters mejora significativamente la precisión del agrupamiento en ambos casos.

4. Si disminuye 2 neuronas o 'K' en su algoritmo, ¿qué variación hay en el agrupamiento?

No se registraron resultados con K=1, por lo que no se cuenta con información sobre el comportamiento de los modelos al reducir el número de grupos. Sin embargo, teóricamente, disminuir K o las neuronas reduce la capacidad del modelo para representar la diversidad de los datos, lo que normalmente degradaría la calidad del agrupamiento.

5. Compare cuál de las dos redes ha proporcionado un mejor resultado desde su propio análisis. Incluya la posición final de los centros obtenida por cada uno de los algoritmos.

Ambos algoritmos generaron resultados muy cercanos, pero K-Means demostró un rendimiento ligeramente superior y una mayor estabilidad (desviación estándar 0.058 frente a 0.0728 en la red competitiva). Su Silhouette Score promedio fue de 0.562, mientras que el de la red fue 0.533, evidenciando una segmentación más consistente.

Centros finales (K=3):

K-Means:

- Cluster 1 → Peso = -6.997, Índice PH = 3.503
- Cluster 2 → Peso = 2.329, Índice PH = -0.192
- Cluster 3 → Peso = -5.500, Índice PH = -3.750

Red Competitiva:

- Neurona 1 → Peso = -9.924, Índice PH = 2.042
- Neurona 2 → Peso = -0.408, Índice PH = 0.609
- Neurona 3 → Peso = -4.076, Índice PH = -6.013

En conclusión, aunque ambas técnicas lograron una correcta separación de los datos, K-Means superó ligeramente a la Red Competitiva tanto en precisión como en estabilidad, ofreciendo agrupamientos más definidos y coherentes.

Código

Ejercicio A

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.gaussian_process import GaussianProcessRegressor

from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

import cv2
```

```
import warnings
warnings.filterwarnings('ignore')


def rbf_image_reconstruction(image_path, damage_percentages=[0.2, 0.6, 0.85]):
    """
    Reconstruye una imagen usando RBF con diferentes porcentajes de daño
    """
    # Cargar imagen y convertir a escala de grises
    img = cv2.imread(image_path)
    if img is None:
        print(f"Error: No se pudo cargar la imagen {image_path}")
        return


    img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    img_gray = img_gray.astype(np.float64) / 255.0


    m, n = img_gray.shape
    print(f"Tamaño de la imagen: {m}x{n} píxeles")


    # Si hay múltiples porcentajes, crear subplots
    if len(damage_percentages) > 1:
        fig, axes = plt.subplots(len(damage_percentages), 3, figsize=(15,
5*len(damage_percentages)))
```

```
else:
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes = [axes]  # Convertir a lista para consistencia

for idx, damage_pct in enumerate(damage_percentages):
    print(f"Procesando {damage_pct*100}% de daño...")

    # Simulación de daño en la imagen
    np.random.seed(42)  # Para resultados reproducibles
    mask = np.random.random((m, n)) > damage_pct
    img_damaged = img_gray.copy()
    img_damaged[~mask] = np.nan

    # Extraer datos de entrenamiento (píxeles no dañados)
    X, Y = np.meshgrid(range(n), range(m))
    coordinates = np.column_stack((X.ravel(), Y.ravel()))
    pixel_values = img_gray.ravel()

    # Filtrar solo píxeles no dañados
    valid_mask = ~np.isnan(img_damaged.ravel())
    X_train = coordinates[valid_mask]
    Y_train = pixel_values[valid_mask]
```

```
print(f"Píxeles válidos para entrenamiento: {len(Y_train)}")


# Reducir número de muestras para evitar problemas de memoria
num_samples = min(5000, len(Y_train))  # Reducido a 5000 para mayor velocidad
if num_samples < len(Y_train):
    indices = np.random.choice(len(Y_train), num_samples, replace=False)
    X_train = X_train[indices]
    Y_train = Y_train[indices]
    print(f"Muestras utilizadas para entrenamiento: {num_samples}")


# Escalar datos
scaler_X = StandardScaler()
scaler_Y = StandardScaler()


X_train_scaled = scaler_X.fit_transform(X_train)
Y_train_scaled = scaler_Y.fit_transform(Y_train.reshape(-1, 1)).ravel()


# Entrenar modelo RBF usando Gaussian Process con kernel RBF
print("Entrenando modelo RBF...")
kernel = C(1.0, (1e-3, 1e3)) * RBF(1.0, (1e-2, 1e2))
gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=3, alpha=0.1)


# Entrenar con subconjunto más pequeño si es necesario
```

```
if len(X_train_scaled) > 2000:
    train_subset = np.random.choice(len(X_train_scaled), 2000, replace=False)
    gp.fit(X_train_scaled[train_subset], Y_train_scaled[train_subset])
else:
    gp.fit(X_train_scaled, Y_train_scaled)

# Predicción de píxeles dañados
damaged_coords = coordinates[~valid_mask]
print(f"Píxeles a reconstruir: {len(damaged_coords)}")

if len(damaged_coords) > 0:
    # Procesar en lotes para evitar memoria insuficiente
    batch_size = 1000
    Y_pred = np.zeros(len(damaged_coords))

    for i in range(0, len(damaged_coords), batch_size):
        end_idx = min(i + batch_size, len(damaged_coords))
        batch_coords = damaged_coords[i:end_idx]

        X_test_scaled = scaler_X.transform(batch_coords)
        Y_pred_scaled = gp.predict(X_test_scaled)
        Y_pred[i:end_idx] = scaler_Y.inverse_transform(Y_pred_scaled.reshape(-1, 1)).ravel()
```

```
    # Reconstruir imagen
    img_restored = img_damaged.copy()
    img_restored_flat = img_restored.ravel()
    img_restored_flat[~valid_mask] = Y_pred
    img_restored = img_restored_flat.reshape((m, n))

    # Asegurar que los valores estén en [0, 1]
    img_restored = np.clip(img_restored, 0, 1)
else:
    img_restored = img_damaged.copy()

# Visualización
if len(damage_percentages) > 1:
    ax_orig, ax_damaged, ax_restored = axes[idx, 0], axes[idx, 1], axes[idx, 2]
else:
    ax_orig, ax_damaged, ax_restored = axes[0], axes[1], axes[2]

ax_orig.imshow(img_gray, cmap='gray')
ax_orig.set_title(f'Imagen Original')
ax_orig.axis('off')

# Para mostrar imagen dañada, reemplazar NaN con 0
img_damaged_display = img_damaged.copy()
```

```
        img_damaged_display[np.isnan(img_damaged_display)] = 0
        ax_damaged.imshow(img_damaged_display, cmap='gray')
        ax_damaged.set_title(f'Dañada ({damage_pct*100:.0f}% píxeles eliminados)')
        ax_damaged.axis('off')

        ax_restored.imshow(img_restored, cmap='gray')
        ax_restored.set_title(f'Restaurada con RBF')
        ax_restored.axis('off')

        print(f"Completado {damage_pct*100}% de daño\n")

    plt.tight_layout()
    plt.show()

# Ejecutar la función
rbf_image_reconstruction('Orb.jpg')
```

Ejercicio B

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

Práctica 4. Redes Neuronales para la reconstrucción con RBF, K-means y redes competitivas.

```
import warnings
warnings.filterwarnings('ignore')


def load_medication_data(file_path):
    """Carga y prepara los datos de medicamentos"""
    data = pd.read_csv(file_path)
    print(f"Datos cargados: {data.shape}")
    print(data.head())


    # Extraer características (Peso e Indice PH)
    X = data[['Peso', 'Indice PH']].values
    return X, data


def kmeans_clustering(X, k=3):
    """Implementa algoritmo K-Means"""
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)
    centers = kmeans.cluster_centers_
    return labels, centers, kmeans


def manual_competitive_clustering(X, n_neurons=3, max_epochs=100):
    """Implementación manual de red competitiva"""
    # Inicializar pesos aleatorios (centros de clusters)
```

```
np.random.seed(42)
centers = np.random.rand(n_neurons, X.shape[1]) * (X.max(axis=0) - X.min(axis=0)) + X.min(axis=0)

learning_rate = 0.01
labels_history = []
centers_history = [centers.copy()]

for epoch in range(max_epochs):
    labels = []
    for i in range(len(X)):
        # Calcular distancias a todos los centros
        distances = np.linalg.norm(X[i] - centers, axis=1)
        # Encontrar la neurona ganadora (más cercana)
        winner = np.argmin(distances)
        labels.append(winner)

        # Actualizar el centro ganador
        centers[winner] += learning_rate * (X[i] - centers[winner])

    labels_history.append(np.array(labels))
    centers_history.append(centers.copy())
```

```
        # Verificar convergencia
        if epoch > 0 and np.all(labels_history[-1] == labels_history[-2]):
            break

    return np.array(labels), centers, len(centers_history)

def compare_clustering_algorithms(file_path):
    """Compara K-Means vs Red Neuronal Competitiva"""

    # Cargar datos
    X, data = load_medication_data(file_path)

    # Escalar datos
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Probar diferentes valores de K/neuronas
    k_values = [2, 3, 4, 5, 7]  # Incluyendo +2 y -2 para comparación
    results = {}

    fig, axes = plt.subplots(2, len(k_values), figsize=(20, 8))

    for i, k in enumerate(k_values):
```

```
print(f"\n--- Analizando con K/Neuronas = {k} ---")

# K-Means
kmeans_labels, kmeans_centers, kmeans_model = kmeans_clustering(X_scaled, k)
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)

# Red Competitiva
comp_labels, comp_centers, comp_iterations = manual_competitive_clustering(X_scaled, k)
comp_silhouette = silhouette_score(X_scaled, comp_labels)

results[k] = {
    'kmeans': {
        'labels': kmeans_labels,
        'centers': scaler.inverse_transform(kmeans_centers),
        'silhouette': kmeans_silhouette,
        'iterations': kmeans_model.n_iter_
    },
    'competitive': {
        'labels': comp_labels,
        'centers': scaler.inverse_transform(comp_centers),
        'silhouette': comp_silhouette,
        'iterations': comp_iterations
    }
```

```
    }

    # Visualización K-Means
    axes[0, i].scatter(data['Peso'], data['Indice PH'], c=kmeans_labels, cmap='viridis', alpha=0.6, s=10)
    axes[0, i].scatter(results[k]['kmeans']['centers'][:, 0],
                       results[k]['kmeans']['centers'][:, 1],
                       marker='X', s=200, c='red', label='Centros', edgecolors='black')
    axes[0, i].set_title(f'K-Means (K={k})\nSilhouette: {kmeans_silhouette:.3f}')
    axes[0, i].set_xlabel('Peso')
    axes[0, i].set_ylabel('Indice PH')
    axes[0, i].legend()

    # Visualización Red Competitiva
    axes[1, i].scatter(data['Peso'], data['Indice PH'], c=comp_labels, cmap='viridis', alpha=0.6, s=10)
    axes[1, i].scatter(results[k]['competitive']['centers'][:, 0],
                       results[k]['competitive']['centers'][:, 1],
                       marker='X', s=200, c='red', label='Centros', edgecolors='black')
    axes[1, i].set_title(f'Red Competitiva (Neuronas={k})\nSilhouette: {comp_silhouette:.3f}')
    axes[1, i].set_xlabel('Peso')
    axes[1, i].set_ylabel('Indice PH')
    axes[1, i].legend()
```

```
plt.tight_layout()
plt.show()

# ANÁLISIS COMPARATIVO MEJORADO
print("\n" + "="*100)
print("ANÁLISIS COMPARATIVO: K-MEANS vs RED NEURONAL COMPETITIVA")
print("="*100)

# 1. ¿Cuántos K o cuantas neuronas fueron necesarias para poder agrupar los datos?
print("\n1. ¿CUÁNTOS K O CUÁNTAS NEURONAS FUERON NECESARIAS PARA PODER AGRUPAR LOS DATOS?")
best_k_kmeans = max(results.items(), key=lambda x: x[1]['kmeans']['silhouette'])[0]
best_k_comp = max(results.items(), key=lambda x: x[1]['competitive']['silhouette'])[0]

print(f"  • K-Means: El mejor resultado se obtuvo con K = {best_k_kmeans} clusters")
print(f"    (Silhouette Score: {results[best_k_kmeans]['kmeans']['silhouette']:.3f})")
print(f"  • Red Competitiva: El mejor resultado se obtuvo con {best_k_comp} neuronas")
print(f"    (Silhouette Score: {results[best_k_comp]['competitive']['silhouette']:.3f})")
print(f"  • Conclusión: Ambos algoritmos funcionan mejor con {best_k_kmeans} clusters/neuronas")

# 2. ¿Cuál de los dos algoritmos convergen más rápido a una misma solución?
```

```
print("\n2. ¿CUÁL DE LOS DOS ALGORITMOS CONVERGE MÁS RÁPIDO A UNA MISMA SOLUCIÓN?")
print("   Comparación de iteraciones para convergencia:")
for k in [3, 5]:
    kmeans_iters = results[k]['kmeans']['iterations']
    comp_iters = results[k]['competitive']['iterations']
    diferencia = kmeans_iters - comp_iters
    mas_rapido = "K-Means" if kmeans_iters < comp_iters else "Red Competitiva"

    print(f"   • K={k}:")
    print(f"     - K-Means: {kmeans_iters} iteraciones")
    print(f"     - Red Competitiva: {comp_iters} iteraciones")
    print(f"     - Más rápido: {mas_rapido} (diferencia: {abs(diferencia)} iteraciones)")

# 3. Si aumenta 2 neuronas o 'k' en su algoritmo, ¿qué variación hay en el agrupamiento?
print("\n3. SI AUMENTA 2 NEURONAS O 'K' EN SU ALGORITMO, ¿QUÉ VARIACIÓN HAY EN EL AGRUPAMIENTO?")
k_ref = 3
k_plus_2 = 5

sil_k3_kmeans = results[k_ref]['kmeans']['silhouette']
sil_k5_kmeans = results[k_plus_2]['kmeans']['silhouette']
sil_k3_comp = results[k_ref]['competitive']['silhouette']
```

```
sil_k5_comp = results[k_plus_2]['competitive']['silhouette']


print(f"  • Comparación K={k_ref} → K={k_plus_2}:")
print(f"    K-Means:")
print(f"      - Silhouette: {sil_k3_kmeans:.3f} → {sil_k5_kmeans:.3f}")
print(f"      - Variación: {sil_k5_kmeans - sil_k3_kmeans:+.3f} ({(sil_k5_kmeans/sil_k3_kmeans - 1)*100:+.1f}%)")
print(f"      - Interpretación: {'MEJORA' if sil_k5_kmeans > sil_k3_kmeans else 'EMPEORA'} la calidad del agrupamiento")


print(f"    Red Competitiva:")
print(f"      - Silhouette: {sil_k3_comp:.3f} → {sil_k5_comp:.3f}")
print(f"      - Variación: {sil_k5_comp - sil_k3_comp:+.3f} ({(sil_k5_comp/sil_k3_comp - 1)*100:+.1f}%)")
print(f"      - Interpretación: {'MEJORA' if sil_k5_comp > sil_k3_comp else 'EMPEORA'} la calidad del agrupamiento")


# 4. Si disminuye 2 neuronas o 'k' en su algoritmo, ¿qué variación hay en el agrupamiento?
print("\n4. SI DISMINUYE 2 NEURONAS O 'K' EN SU ALGORITMO, ¿QUÉ VARIACIÓN HAY EN EL AGRUPAMIENTO?")
k_minus_2 = 1


if k_minus_2 in results:
    sil_k1_kmeans = results[k_minus_2]['kmeans']['silhouette']
```

```
        sil_k1_comp = results[k_minus_2]['competitive']['silhouette']


        print(f"   • Comparación K={k_ref} → K={k_minus_2}:")
        print(f"     K-Means:")
        print(f"       - Silhouette: {sil_k3_kmeans:.3f} → {sil_k1_kmeans:.3f}")
        print(f"       - Variación: {sil_k1_kmeans - sil_k3_kmeans:+.3f} ({(sil_k1_kmeans/sil_k3_kmeans - 1)*100:+.1f}%)")
        print(f"       - Interpretación: {'MEJORA' if sil_k1_kmeans > sil_k3_kmeans else 'EMPEORA'} la calidad del agrupamiento")


        print(f"     Red Competitiva:")
        print(f"       - Silhouette: {sil_k3_comp:.3f} → {sil_k1_comp:.3f}")
        print(f"       - Variación: {sil_k1_comp - sil_k3_comp:+.3f} ({(sil_k1_comp/sil_k3_comp - 1)*100:+.1f}%)")
        print(f"       - Interpretación: {'MEJORA' if sil_k1_comp > sil_k3_comp else 'EMPEORA'} la calidad del agrupamiento")
    else:
        print(f"   • No se tiene datos para K=1 en esta ejecución")


    # 5. Compare cuál de las dos redes ha proporcionado un mejor resultado
    print("\n5. COMPARE CUÁL DE LAS DOS REDES HA PROPORCIONADO UN MEJOR RESULTADO")


    # Calcular el mejor silhouette promedio
```

```
avg_sil_kmeans = np.mean([results[k]['kmeans']['silhouette'] for k in k_values])
avg_sil_comp = np.mean([results[k]['competitive']['silhouette'] for k in k_values])

best_overall_kmeans = max([results[k]['kmeans']['silhouette'] for k in k_values])
best_overall_comp = max([results[k]['competitive']['silhouette'] for k in k_values])

print(f"  • Métricas de comparación:")
print(f"    - Mejor Silhouette Score:")
print(f"      * K-Means: {best_overall_kmeans:.3f} (K={best_k_kmeans})")
print(f"      * Red Competitiva: {best_overall_comp:.3f} (Neuronas={best_k_comp})")
print(f"    - Silhouette Score Promedio:")
print(f"      * K-Means: {avg_sil_kmeans:.3f}")
print(f"      * Red Competitiva: {avg_sil_comp:.3f}")

if best_overall_kmeans > best_overall_comp:
    print(f"  • CONCLUSIÓN: K-MEANS proporciona MEJORES resultados")
    mejor_algoritmo = "K-Means"
    ventaja = best_overall_kmeans - best_overall_comp
else:
    print(f"  • CONCLUSIÓN: RED COMPETITIVA proporciona MEJORES resultados")
    mejor_algoritmo = "Red Competitiva"
    ventaja = best_overall_comp - best_overall_kmeans
```

```
    print(f"     (Ventaja: {ventaja:.3f} en Silhouette Score)")


    # 6. Posición final de los centros obtenida por cada algoritmo (para K=3)
    print("\n6. POSICIÓN FINAL DE LOS CENTROS OBTENIDA POR CADA ALGORITMO (K=3)")


    print(f"   • K-MEANS - Centros finales (K=3):")
    for i, center in enumerate(results[3]['kmeans']['centers']):
        print(f"     Cluster {i+1}: Peso = {center[0]:.3f}, Índice PH = {center[1]:.3f}")


    print(f"   • RED COMPETITIVA - Centros finales (3 Neuronas):")
    for i, center in enumerate(results[3]['competitive']['centers']):
        print(f"     Neurona {i+1}: Peso = {center[0]:.3f}, Índice PH = {center[1]:.3f}")


    # Análisis adicional de estabilidad
    print("\n7. ANÁLISIS ADICIONAL - ESTABILIDAD DE LOS ALGORITMOS")


    # Calcular desviación estándar de silhouette scores
    std_kmeans = np.std([results[k]['kmeans']['silhouette'] for k in k_values])
    std_comp = np.std([results[k]['competitive']['silhouette'] for k in k_values])


    print(f"   • Estabilidad (desviación estándar de Silhouette Scores):")
    print(f"     - K-Means: {std_kmeans:.4f}")
```

```
    print(f"     - Red Competitiva: {std_comp:.4f}")
    print(f"     - Más estable: {'K-Means' if std_kmeans < std_comp else 'Red Competitiva'}")

    return results, mejor_algoritmo

# Ejecutar análisis
if __name__ == "__main__":
    print("INICIO DEL ANÁLISIS DE AGRUPAMIENTO")
    print("="*50)
    results, mejor_algoritmo = compare_clustering_algorithms('datos_medicamentos2025A.csv')
    print("\n" + "="*50)
    print(f"ANÁLISIS COMPLETADO - ALGORITMO RECOMENDADO: {mejor_algoritmo}")
    print("="*50)
```

Conclusión

Los resultados obtenidos demostraron que tanto las redes neuronales como los métodos de agrupamiento no supervisado poseen un alto potencial en tareas de aprendizaje sin etiquetas. En el primer ejercicio, la red RBF logró reconstruir imágenes con gran precisión incluso con altos niveles de daño, mostrando una alta capacidad de generalización y suavización de patrones, aunque limitada por el costo computacional. En el segundo ejercicio, tanto K-Means como la Red Competitiva lograron agrupar correctamente los datos, destacando K-Means por su mayor estabilidad, rapidez y precisión en los resultados. En conjunto, las prácticas permitieron comprender cómo diferentes arquitecturas neuronales pueden aplicarse efectivamente a problemas complejos, desde la restauración visual hasta la clasificación automática, resaltando la importancia

del análisis comparativo para elegir el modelo más adecuado según el tipo de datos y el objetivo propuesto.

En lo personal esta practica me costo y bastante… mas que nada porque las librerías que utilicé para Python me dieron errores y tuve que desinstalarlas, actualizarlas o simplemente cambiar la versión. específicamente en el problema de la imagen, en reiteradas ocasiones me salió que tenia problemas de memoria RAM así que tuve que buscar alternativas (ejem DeepSeek me ayudo con eso y chatgpt a los comentarios)

Bibliografía

Ruz Canul, M. A. (s.f.). Redes Neuronales Artificiales y Aprendizaje profundo. *Redes Neuronales Artificiales.* Centro Universitario de Ciencias Exactas e Ingenierías, Universidad de Guadalajara. https://classroom.google.com/u/1/c/Nzk1Mjc3MTQ5MjIy/m/ODAwMTA4MDM3NzIx/details